```
// Printing the result.
        List<Integer> result = new ArrayList<>();
        while(hash[lastIdx] != lastIdx){
            result.add(nums[lastIdx]);
            lastIdx = hash[lastIdx];
        }
        result.add(nums[lastIdx]); // add the root element
        Collections.reverse(result);

        return result;

    }
}
```

Binary Trees:

Traversal in Binary Trees:

- BFS - Breadth First Search / Level Wise Traversal
- DFS - Depth First Search
    - Inorder → `Left-Root-Right`
    - Preorder → `Root-Left-Right`
    - Post-order → `Left-Right-Root`

Preorder Traversal

- Recursive
  TC - O(N)
  SC- O(N) Stack Space

```
import java.util.* ;
import java.io.*;

/*********************************************************
Following is the BinaryTreeNode class structure

class BinaryTreeNode<T> {
    T data;
    BinaryTreeNode<T> left;
    BinaryTreeNode<T> right;

    public BinaryTreeNode(T data) {
        this.data = data;
    }
}
*********************************************************/
import java.util.ArrayList;

public class Solution {
    public static ArrayList<Integer> preorderTraversal(BinaryTreeNode<Integer> root) {
        // Write your code here
        ArrayList<Integer> list = new ArrayList<>();
        findSolution(root, list);
```

```
        return list;

    }

    public static void findSolution(BinaryTreeNode<Integer> root, ArrayList<Integer> list){

        if(root == null)
            return ;

        int data = root.data;
        list.add(data);
        findSolution(root.left, list);
        findSolution(root.right, list);
    }
}
```

- Iterative

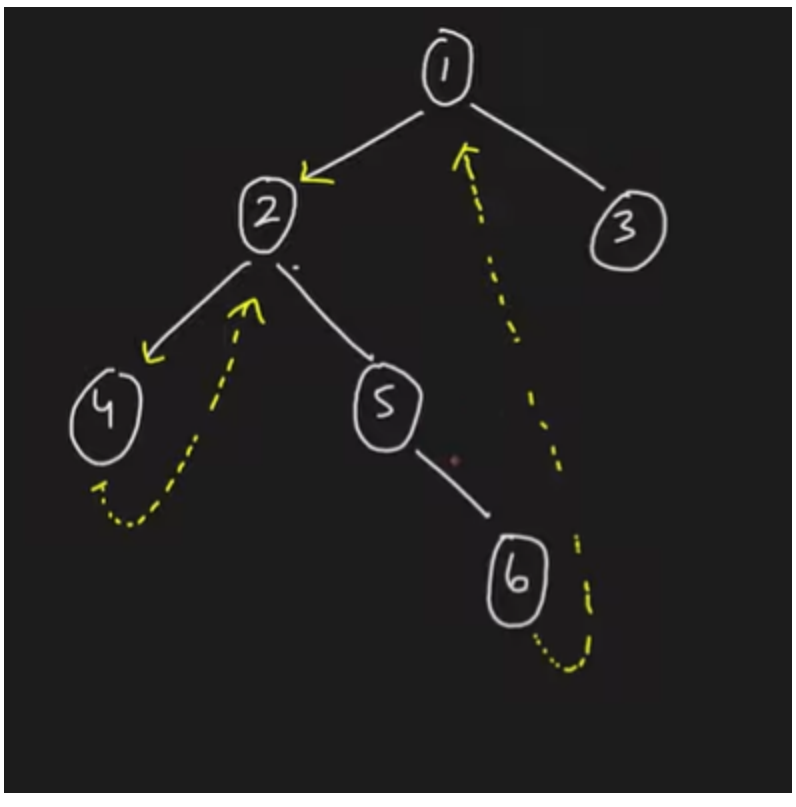
  Preorder → Root-Left-Right

  Algorithm:
  1. Create STACK - stores the order of Execution
  2. If Right child exists then push that first and then left, since we want to process left child first

  TC - O(N)
  SC- O(N) Stack Space

```
public class Solution {
    public static ArrayList<Integer> preorderTraversal(BinaryTreeNode<Integer> root) {
        // Write your code here
        ArrayList<Integer> list = new ArrayList<>();
        Stack<BinaryTreeNode<Integer>> st = new Stack<>();

        st.push(root);

        while(!st.empty()){
            BinaryTreeNode<Integer> node = st.peek();
            list.add(node.data);
            st.pop();

            if(node.right != null)
                st.push(node.right);
            if(node.left != null)
                st.push(node.left);
        }
        return list;
    }
}
```

Special: **MORRIS TRAVERSAL : Threaded Binary Tree concept.**
TC: O(N)
SC: O(1)*



Intuition: *From leaf node directly go to root.*

CASE 1:
When left child is not there → print node and move right
CASE 2:
When left child exists the last leaf of right child points to root.

How we know we came twice here from bottom leaf?
→ if already thread exists then visiting the node 2nd time.

```
 //MORRIS TRAVERSAL
 //if left child null then add node move right
 // else connect the thread of rightmost leaf to parent if visiting first time and move left,
 //else add to the solution and move to the right.

class Solution {
    public List<Integer> preorderTraversal(TreeNode root) {

        List<Integer> ans = new ArrayList<>();
        TreeNode curr = root;
        while(curr != null){
            if(curr.left == null){
                ans.add(curr.val);
                curr = curr.right;
            } else {
                //find rightmost leaf in left subtree
                TreeNode predecessor = curr.left;
                while(predecessor.right != null && predecessor.right != curr)  predecessor =
predecessor.right;

                //visiting for the first time --> add thread to root.
                if(predecessor.right == null) {
                    ans.add(curr.val);
                    predecessor.right = curr;
                    curr = curr.left;
                }
                //visiting 2nd time
                else{
                    predecessor.right = null;
                    curr = curr.right;
                }
            }
```

```
        }
        return ans;
    }
}
```

Inorder Traversal

Morris Traversal:
TC- O(N)
SC- O(1)* → that's special

```
class Solution {
    //  MORRIS Inorder
    // if left-child: null --> add to ans, move to right
    // else add thread to rightleaf of left subtree.
    public List<Integer> inorderTraversal(TreeNode root) {
        List<Integer> ans = new ArrayList<Integer>();
        TreeNode curr = root;
        while(curr != null){
            if(curr.left == null){
                ans.add(curr.val);
                curr = curr.right;
            } else{
                TreeNode rightLeaf = curr.left;
                while(rightLeaf.right != null && rightLeaf.right != curr)   rightLeaf = rightLeaf.right;

                //if visiting first time
                if(rightLeaf.right == null){
                    rightLeaf.right = curr;
                    curr = curr.left;
                } else{
                    //visiting node second time
                    ans.add(curr.val); //add to list
                    rightLeaf.right = null;
                    curr = curr.right;
                }
            }
        }
        return ans;
    }
}
```

Level Order Traversal

1. **Iterative**:
   Maintain a Queue to store elements level wise.

```
class Solution {
    public List<List<Integer>> levelOrder(TreeNode root) {

        List<List<Integer>> ans = new ArrayList<>();
        if(root == null)    return ans;

        Queue<TreeNode> queue = new LinkedList<>();
        queue.add(root);
```

```
        while(!queue.isEmpty()){
            int levelCount = queue.size();
            List<Integer> levelNodes = new ArrayList<>();
            while(levelCount  != 0){
                TreeNode curr = queue.remove();
                levelNodes.add(curr.val);

                if(curr.left != null)   queue.add(curr.left);
                if(curr.right != null)  queue.add(curr.right);
                levelCount--;
            }
            ans.add(levelNodes);
        }

        return ans;

    }
}
```

2. **Recursive**:
   Add nodes to each level and follow recursion.

```
class Solution {
    public List<List<Integer>> levelOrder(TreeNode root) {

        List<List<Integer>> ans = new ArrayList<>();
        if(root == null)    return ans;

        traverseTree(root, 0, ans);
        return ans;
    }

    void traverseTree(TreeNode node, int level, List<List<Integer>> list){
        if(node == null)
            return;

        if(list.size() <= level)
            list.add(new ArrayList<Integer>());

        list.get(level).add(node.val);
        traverseTree(node.left, level+1,list);
        traverseTree(node.right, level+1, list);
    }
}
```

Q1. **Get all three traversal in one iteration**
   Store a Pair, where you store how many times you visit the node
   The same logic which you apply to solve in paper.

```
class myPair {
    TreeNode t;
    Integer n;

    myPair(TreeNode t, Integer n) {
        this.t = t;
        this.n = n;
    }
}

class Solution {
    List<List<Integer>> treeTraversal(TreeNode root) {
        List<List<Integer>> ans = new ArrayList<>();
```

```java
        List<Integer> preOrder = new ArrayList<>();
        List<Integer> inOrder = new ArrayList<>();
        List<Integer> postOrder = new ArrayList<>();

        if (root == null) {
            // Return empty traversal lists if the tree is empty
            return Arrays.asList(preOrder, inOrder, postOrder);
        }

        Stack<myPair> st = new Stack<>();
        myPair rootNode = new myPair(root, 1);
        st.add(rootNode);

        while (!st.isEmpty()) {
            myPair curr = st.pop();

            if (curr.n == 1) {
                // PreOrder traversal
                preOrder.add(curr.t.val);
                curr.n = 2;
                st.push(curr);

                // Add left child
                if (curr.t.left != null) {
                    st.add(new myPair(curr.t.left, 1));
                }
            } else if (curr.n == 2) {
                // InOrder traversal
                inOrder.add(curr.t.val);
                curr.n = 3;
                st.push(curr);

                // Add right child
                if (curr.t.right != null) {
                    st.add(new myPair(curr.t.right, 1));
                }
            } else {
                // PostOrder traversal
                postOrder.add(curr.t.val);
            }
        }

        // Add traversals to the result in the desired order
        ans.add(preOrder);  // PreOrder
        ans.add(inOrder);   // InOrder
        ans.add(postOrder); // PostOrder

        return ans;
    }
}
```

Q2. **Height of Binary Tree:**

Can be done in 2 ways:

- Level order traversal - Iterative, get the max level
- DFS - Recursion

Recursion:

```java
class Solution {
    public int maxDepth(TreeNode root) {
        if(root == null) return 0;

        int height=calcDepth(root, 1);
        return height;
    }

    int calcDepth(TreeNode node, int height){
        if(node == null)
            return height;
```

```
        if(node.left == null && node.right == null)
            return height;

        return Math.max(calcDepth(node.left, height+1), calcDepth(node.right, height+1));
    }
}
```

Q3. **Check balanced binary tree** (diff b/w height of leftSubtree and rightSubtree <=1)

Solution: $O(n^2)$

```
class Solution {
    public boolean isBalanced(TreeNode root) {
        if(root==null)
            return true;

        int leftHeight = root.left == null ? 0 : calcHeight(root.left, 1);
        int rightHeight = root.right == null ? 0 : calcHeight(root.right, 1);
        System.out.println(leftHeight + " : " + rightHeight);
        if(Math.abs(leftHeight-rightHeight) > 1)
            return false;

        return isBalanced(root.left) && isBalanced(root.right);  //Check for below subtrees also,
                                                                 // If this is NOT done, only checking root node → WRONG ANSWER
                                                                 [1,2,2,3,null,null,3,4,null,null,4]
    }

    int calcHeight(TreeNode node, int level){
        if(node==null)
            return level;
        if(node.left==null && node.right==null)
            return level;

        return Math.max(calcHeight(node.left, level+1), calcHeight(node.right, level+1));
    }
}
```

Method2: $O(n)$

Algo : Recursive search in left and right and store the result
If any subtree

```
class Solution {
    public boolean isBalanced(TreeNode root) {
        // Use a helper function to check balance and compute height
        return checkHeight(root) != -1;
    }

    private int checkHeight(TreeNode node) {
        if (node == null) {
            return 0; // Height of an empty subtree is 0
        }
        int leftHeight = checkHeight(node.left);
        if (leftHeight == -1) {
            return -1; // Left subtree is unbalanced
        }
```

```
        int rightHeight = checkHeight(node.right);
        if (rightHeight == -1) {
            return -1; // Right subtree is unbalanced
        }
        // Check if the current node is balanced
        if (Math.abs(leftHeight - rightHeight) > 1) {
            return -1; // Current node is unbalanced
        }
        // Return the height of the current subtree
        return Math.max(leftHeight, rightHeight) + 1;
    }
}
```

Q4. Diameter of Binary Tree : **Distance b/w any 2 leaf nodes** (need not be passing by root)

```
class Solution {
    public int diameterOfBinaryTree(TreeNode root) {
        int[] result = new int[1];
        traverseTree(root, result);

        return result[0];
    }

    int traverseTree(TreeNode node, int[] result){
        if(node == null)
            return 0;

        int leftHeight=0, rightHeight=0;

        if(node.left != null)
            leftHeight = traverseTree(node.left, result);

        if(node.right != null)
            rightHeight = traverseTree(node.right, result);

        //no. of edges = No of nodes - 1;
        //no. of edges= [leftHeight + rightHeight + 1(root)] - 1;
        result[0] = Math.max(result[0], (leftHeight + rightHeight));

        return Math.max(leftHeight, rightHeight) + 1;
    }
}
```

Q5. Maximum Sum Path in Binary Tree-

Good Question:

```
class Solution {
    public int maxPathSum(TreeNode root) {
        int[] maxSum = {Integer.MIN_VALUE};
        traverseTree(root, maxSum);

        return maxSum[0];
    }

    int traverseTree(TreeNode node, int[] maxSum){
        if(node == null)
            return 0;
```

```
        int leftTreeSum = traverseTree(node.left, maxSum);
        int rightTreeSum = traverseTree(node.right, maxSum);

        maxSum[0] = Math.max(maxSum[0],
                        Math.max(node.val, //consider only nodes if left and right sum negative.
                            Math.max(leftTreeSum + rightTreeSum + node.val, //all nodes
                                Math.max(leftTreeSum + node.val,  //left subtree
                                         rightTreeSum + node.val))));  //right subtree

        return Math.max(node.val,   //node itself if left and right child is negative.
                            Math.max(node.val+leftTreeSum, node.val+rightTreeSum));

    }
}
```

OR – better logic → *ignore negative sum from left and right subtrees during function calls only* .

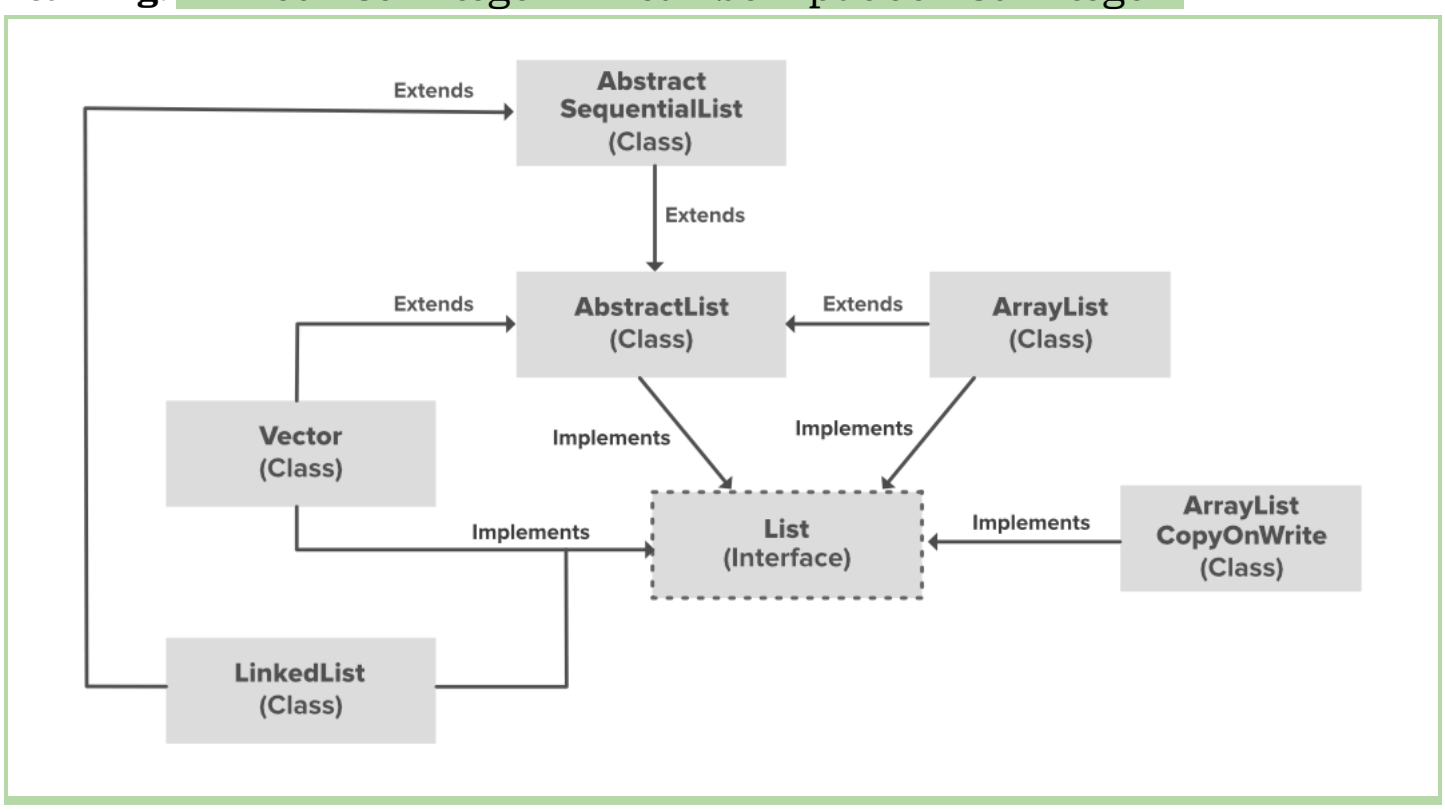
```
int traverseTree(TreeNode node, int[] maxSum){
        if(node == null)
            return 0;

        int leftTreeSum = Math.max(0, traverseTree(node.left, maxSum));
        int rightTreeSum = Math.max(0,traverseTree(node.right, maxSum));

        maxSum[0] = Math.max(maxSum[0], leftTreeSum + rightTreeSum + node.val);

        return Math.max(leftTreeSum, rightTreeSum) + node.val;

    }
```

Q6. **Zig - Zag Traversal:**

```
class Solution {
    public List<List<Integer>> zigzagLevelOrder(TreeNode root) {
        List<List<Integer>> result = new ArrayList<>();
        if(root == null)
            return result;

        Queue<TreeNode> queue = new LinkedList<>();
        queue.add(root);

        boolean leftToRight = true;

        while(!queue.isEmpty()){

            List<Integer> levelList = new LinkedList<>();
            int levelSize = queue.size();

            for(int i = 0; i < levelSize ; i++){

                TreeNode node = queue.peek();
                queue.remove();

                if(leftToRight)
                    levelList.addLast(node.val);
                else
                    levelList.addFirst(node.val);

                if(node.left != null)   queue.add(node.left);
```

```
                if(node.right != null)  queue.add(node.right);
            }

            result.add(levelList);
            leftToRight = (!leftToRight);

        }

        return result;
    }

}
```

Learning: LinkedList<Integer> → can be input ot List<Integer>



Q7. Boundary Traversal – Good question

```
public class Solution {
    public static List<Integer> traverseBoundary(TreeNode root) {
        List<Integer> result = new ArrayList<>();
        if (root == null)
            return result;

        // Add root node if it's not a leaf
        if (!isLeaf(root))
            result.add(root.data);

        // Add left boundary (excluding leaf nodes)
        fetchLeftBoundary(root.left, result);

        // Add all leaf nodes
        fetchLeafNodes(root, result);

        // Add right boundary (excluding leaf nodes and root)
        List<Integer> rightBoundary = new ArrayList<>();
        fetchRightBoundary(root.right, rightBoundary);
        Collections.reverse(rightBoundary);
        result.addAll(rightBoundary);

        return result;
    }

    private static boolean isLeaf(TreeNode node) {
```

```java
        return node.left == null && node.right == null;
    }

    private static void fetchLeftBoundary(TreeNode node, List<Integer> result) {
        while (node != null) {
            if (!isLeaf(node))
                result.add(node.data);

            // Traverse left or fallback to right
            node = (node.left != null) ? node.left : node.right;
        }
    }

    private static void fetchLeafNodes(TreeNode node, List<Integer> result) {
        if (node == null)
            return;

        if (isLeaf(node)) {
            result.add(node.data);
        } else {
            fetchLeafNodes(node.left, result);
            fetchLeafNodes(node.right, result);
        }
    }

    private static void fetchRightBoundary(TreeNode node, List<Integer> result) {
        while (node != null) {
            if (!isLeaf(node))
                result.add(node.data);

            // Traverse right or fallback to left
            node = (node.right != null) ? node.right : node.left;
        }
    }
}
```

Vertical Order Traversal

My approach:

Why this is **WRONG** :

Why Sort Nodes Within Each Level Before Merging?

In vertical traversal, the order of nodes in the final answer depends on two criteria:

1. **Vertical Order (Horizontal Levels)**:
   - Nodes at different horizontal levels are separated into distinct groups.
   - This is handled by organizing nodes into a `TreeMap` or two separate maps (`leftMap` and `rightMap`) as in your original code.

2. **Within the Same Vertical Level**:
   - When multiple nodes exist at the same vertical level, their relative order must be determined by:
     - **Vertical position (row or depth): Nodes higher in the tree come first.**
     - **Value order (tie-breaking): When multiple nodes exist at the same vertical and horizontal position, they must be sorted by their value.**

Why Sorting at Each Level Is Necessary?

Sorting within each vertical level ensures that:

1. **Nodes in Higher Rows Come First**:
   - For example, if two nodes `A` and `B` are at the same horizontal and vertical level, the node closer to the root (smaller depth) should come first in the result.
2. **Stable Tie-Breaking**:
   - ***If multiple nodes exist at the same depth and level, their values are sorted for deterministic output.***